FIELD GUIDE / FREE PREVIEW

Web Cache Poisoning & Deception Explained

Know what gets stored, why requests match, and who receives the response

01	02	03	04
REQUEST	MATCH	STORAGE	REUSE

VISUAL EBOOK / LOCAL HTTP PRACTICE

Haroon Shahid

Expanded edition • November 2026

A cache is a promise

You request a public catalog. The page is wrong because an earlier request changed the copy. Now imagine that the retained page contains someone else’s account note. These problems can use the same infrastructure, but they do not fail in the same way.

This book follows one fictional shop, Orchard Books. Its public catalog has useful variants. Alice and Bob have different private notes. You will learn to tell healthy reuse, wrong public reuse and private exposure apart, then verify the repair without sacrificing intended caching.

Begin with requests and identity, then learn matching, admission and time. After that, trace poisoning and deception separately. Finish with evidence, architecture, remediation and a capstone. No Python is required for reading. The optional lab uses Python only to run a local website.

For each chapter, study the diagram and try both questions before opening the answers. In the workbook, predict the clean actor’s result, run a comparison, and write what the evidence does and does not support. After a week, use a different fictional route.

01	REQUEST	What method, target, fields and credentials arrived at this layer?
02	MATCH	Which requests can select the same representation, and why?
03	STORAGE	What permits this exact response to be retained here?
04	REUSE	Who may receive it, when must it be validated, and how is it removed?

Practice only on systems you own or have explicit authorization and isolation to assess. Use synthetic roles and harmless markers. This is educational material, not a production cache, vendor certification or guarantee of professional results.

Contents

I / Why caches exist

01. One copy, two readers 4

02. Requests, responses and URL parts 5

03. Why latency changes architecture 6

This preview contains the introduction and Chapters 1–3. The full book continues through caching semantics, poisoning, deception, evidence and remediation.

01. One copy, two readers



A cache keeps a result so the next request can avoid repeating work. Think of a library desk keeping a copy of its opening hours. Anyone can use that copy. A borrower's account note looks like another piece of paper, but the desk cannot hand it to the next visitor. Speed comes from sharing; safety comes from knowing what may be shared.

Orchard Books will be our fictional shop. Its catalog has English and French versions. Its account screen contains a different note for each customer. Ask four questions whenever a response passes through a cache: What request reached this layer? Which requests match? What may be stored? Who may get the copy, and for how long?

A cache hit only tells you that a stored result was used at some layer. It does not prove the response was appropriate or that a vulnerability exists. For the catalog, a hit returning the expected public version is a healthy control. For an account page, a hit containing another person's marker can be evidence of a broken access boundary.

CHECK YOUR UNDERSTANDING

1.1 Does a hit alone prove a bug?

1.2 Can two correct public responses differ?

1.3 A public book description and an account note have the same status and byte length. What property separates them?

02. Requests, responses and URL parts

01 Request	02 Handler	03 Response	04 Cache
Method + target + fields	Chooses a resource	Status + fields + body	Evaluates its own rules

A request asks the server to do something. The method names the operation, the target identifies what it acts on, and fields can carry preferences or credentials. A response has a status, fields and usually a body. The status alone does not identify the body: a 200 might be a catalog, an account page or an HTML fallback.

In https://shop.example/catalog?lang=fr#details, the scheme is https, the host is shop.example, the path is /catalog, and the query is lang=fr. A normal browser does not send the fragment #details as part of the HTTP target. A field named Accept-Language is separate from the query even if both can select French.

TLS protects traffic between two endpoints of a connection. An intermediary that terminates TLS can read the request and store its response. Do not confuse encrypted transport with a privacy rule for a shared cache.

Write down the target and fields actually transmitted. An address bar, a tool editor and an origin log may show different representations of one journey.

CHECK YOUR UNDERSTANDING

2.1 Does the fragment normally reach the HTTP cache?

2.2 Can HTTPS responses be cached?

2.3 A URL fragment differs between two browser navigations. Should an HTTP edge automatically see two different targets?

03. Why latency changes architecture

01 Distance
Round trips have a floor

02 Work
Origin pays per request

03 Reuse
Skip repeated work nearby

04 Trade-off
Copies introduce state

Latency is how long one operation takes. Throughput is how many operations the system completes per unit of time. A customer can wait 300 milliseconds while a service still handles thousands of requests per second because many are in progress together. A cache can help both, but the two improvements are different. A close copy avoids distant trips; a shared copy can save origin CPU and database work.

Imagine 1,000 catalog requests per second. If 950 are served by an edge and the other 50 go to origin, the edge hit ratio is 95% and origin receives 50 requests per second from that stream. If a purge turns every request into a miss for a moment, the origin may suddenly see up to 20 times that rate unless request collapsing, shielding or admission controls change the workload. This is why engineers avoid casually disabling all caching during a repair.

The security question is which work the architecture skipped. A public catalog can skip rendering. A private account cannot skip an access decision just because rendering is costly. Measure cacheable traffic separately from bypassed routes. A global hit ratio may hide a broken account policy or a slow uncached API. Review median and tail latency, hit ratio and origin load together.

CHECK YOUR UNDERSTANDING

3.1 At a 95% hit ratio, how many of 1,000 requests per second reach origin in the simple model?

3.2 Why can one purge cause a load spike?

3.3 What should a private-route repair preserve?

Try the complete method

The complete edition contains 80 illustrated chapters and 240 explained checks. The optional Orchard Cache Observatory includes 38 fixed local HTTP experiments and comparisons. The workbook guides each run and adds 32 fictional review cases.

Start with a healthy miss, store and hit. Then compare wrong public reuse with private retention. The capstone asks you to repair the two causes separately and verify old objects and safe refill.

Published by Haroon Shahid. Prepared with AI assistance. Individual learning license; no resale or redistribution of the paid publication. The unchanged free preview may be shared with attribution. The optional lab is an original teaching implementation. For help or corrections, reply to your Gumroad receipt or email gaknt2@gmail.com.